

Datetime operations, indexing, and slicing

Jayita Chakraborty (PhD)

Date and time objects in R

The base package, one of R's core packages, provides two types of date and time classes:

1. **Date**: This is a simple representation of a calendar date following the **ISO 8601** international standard format (or the Gregorian calendar format) using the **YYYY-m-d** date format. Each date object has a numeric value of the number of days since the origin point (the default setting is 1970-01-01).
2. **POSIXct/POSIXlt**: Also known as the **DateTime** classes (that is, they represent both date and time), these are two POSIX date/time classes that represent the calendar date, the time of the day, and the time zone using the **ISO 8601** international standard format of **YYYY-m-d H:M:S**. For **POSIXct**, "ct" stand for calendar time, it stores the number of seconds since the origin. On the other hand, for **POSIXlt** "lt", or local time, keeps the date as a list of time attributes (such as "hour" and "mon"). It is best to use **POSIXct** as the list structure of a date in **POSIXlt** can be troublesome and **POSIXct** takes up quite a bit less memory.

```
# We will use the Sys.Date and Sys.time functions to pull  
# date and time objects respectively
```

```
date <- Sys.Date()  
time_ct <- Sys.time()
```

```
date
```

```
## [1] "2026-07-16"
```

```
time_ct
```

```
## [1] "2026-07-16 19:25:16 CEST"
```

```
# Let's check the class of the two objects:  
class(date)
```

```
## [1] "Date"
```

```
class(time_ct)
```

```
## [1] "POSIXct" "POSIXt"
```

By default, the `Sys.time` function returns an object of the `POSIXct`, `POSIXt`, or `POSIXct` class. We can use the `as.POSIXlt` function to convert the object to a `POSIXlt` object. While both the `POSIXct` and `POSIXlt` objects have the same representation, the key difference lies in the method in which each of them are stored internally. We will use the `unclass` command to investigate the internal format.

```
time_lt <- as.POSIXlt(time_ct)
```

```
class(time_lt)
```

```
## [1] "POSIXlt" "POSIXt"
```

```
unclass(time_ct)
```

```
## [1] 1784222716
```

```
unclass(time_lt)
```

```
## $sec
```

```
## [1] 16.31704
```

```
##
```

```
## $min
```

```
## [1] 25
```

```
##
```

```
## $hour
```

```
## [1] 19
```

```
##
```

```
## $mday
```

```
## [1] 16
```

```
##
```

```
## $mon
```

```
## [1] 6
```

```
##
```

```
## $year
```

```
## [1] 126
```

```
##
```

```
## $wday
```

```
## [1] 4
```

```
##
```

```
## $yday
```

```
## [1] 196
```

```
##
```

```
## $isdst
```

```
## [1] 1
```

```
##
```

```
## $zone
```

```
## [1] "CEST"
```

```
##
```

```
## $gmtoff
```

```
## [1] 7200
```

```
##
```

```
## attr(,"tzone")
```

```
## [1] "" "CET" "CEST"
```

```
## attr(,"balanced")
```

```
## [1] TRUE
```

Creating date and time objects

We can convert the “2014-5-12” string into a Date object using the `as.Date` function. This is feasible due to the intrinsic characteristics of R that can assign values to a specific class or object, which is usually the combination of `as.[the class name]`.

```
date <- as.Date("2014-5-12")
time_ct <- as.POSIXct("2014-5-12 20:05:35", tz = "EST")

date_time_ct <- as.Date("2014-5-12 20:05:35")

date
```

```
## [1] "2014-05-12"
```

```
time_ct
```

```
## [1] "2014-05-12 20:05:35 EST"
```

```
date_time_ct
```

```
## [1] "2014-05-12"
```

```
class(date)
```

```
## [1] "Date"
```

```
class(time_ct)
```

```
## [1] "POSIXct" "POSIXt"
```

```
class(date_time_ct)
```

```
## [1] "Date"
```

If the format of the input object is different from the **ISO 8601** standard, using the `as.Date` function without declaring the object format structure would return incorrect results. For example, let's try to convert the 31-01-2018 date to Date format without reformatting it.

```
as.Date("31/01/2018")
```

```
## [1] "0031-01-20"
```

In order to solve this issue, you will need to map the different components of the input object to the structure of the Date object.

```
as.Date("31/01/2018", format = "%d/%m/%Y")
```

```
## [1] "2018-01-31"
```

The following figure 1 provides a summary of the main arguments for date objects. You can also checkout this [documentation](#) for your reference.

Date objects are stored internally in R as numeric values that represent the number of days from the origin point. You can easily retrieve the numeric value of the object by using the `as.numeric` function.

Symbol	Meaning	Example
%a	Abbreviated weekday name in the current locale	Sun, Mon
%A	Full weekday name in the current locale	Sunday, Monday
%b	Abbreviated month name	Jan, Feb, Mar
%B	Full month name	January, February
%d	Day of the month as a decimal number	01, 02, 03
%m	Month as a decimal number	01, 02, 03
%y	A year without a century (two-digit)	18
%Y	A year with a century (four-digit)	2018
%r	Equivalent to %I:%M:%S %p for 12-hour clock time	9:35:00 PM
%H	Hours as a decimal number (00-23)	17:00:00
%I	Hours as a decimal number (01-12)	5:00:00 PM
%M	Minutes as a decimal number (00-59)	17:15:00
%S	Seconds as an integer (00-61), allowing for up to two leap-seconds	17:15:45
%p	AM/PM indicator in the locale	AM, PM
%T	Equivalent to %H:%M:%S	17:15:00
%Z	Time zone abbreviation as a character string	GMT, EST, PDT

Figure 1: Symbols used for formatting date.

```
as.numeric(as.Date("31-01-2018", format = "%d-%m-%Y"))
```

```
## [1] 17562
```

The numeric value represents the number of seconds between the ‘origin’ of R and the the date specified. Now, let’s say you have some data imported from excel which is of the following numeric format:

```
dates_excel <- c(42755, 42756, 42757, 42758, 42759, 42760)
```

Now, Excel uses a different origin point than R. Therefore, we will have to add the origin argument and specify the original date that is used to generate the data.

```
dates_excel_no_origin <- as.Date(dates_excel)
dates_excel_origin_added <- as.Date(dates_excel, origin = as.Date("1899-12-30"))
```

```
dates_excel_no_origin
```

```
## [1] "2087-01-22" "2087-01-23" "2087-01-24" "2087-01-25" "2087-01-26"
## [6] "2087-01-27"
```

```
dates_excel_origin_added
```

```
## [1] "2017-01-20" "2017-01-21" "2017-01-22" "2017-01-23" "2017-01-24"
## [6] "2017-01-25"
```

The following table 1 provides some examples for the origin point of commonly used software:

Table 1: Origin dates used by common software packages.

Software	Origin Date
Excel	12/30/1899
Python	1/1/1970
R	1/1/1970
SAS	1/1/1960
Stata	1/1/1960

Now, let's work with some examples to conduct datetime manipulations in the real world data. First we will load some packages which will be useful.

```
require(tidyverse)
require(readxl)
```

We will be using the stock exchange price dataset from the [UCI Machine Learning Repository](#).

```
stock_exchange <- read_excel("./data/data_akbilgic.xlsx", skip = 1)

knitr::kable(head(stock_exchange))
```

date	ISE...2	ISE...3	SP	DAX	FTSE	NIKKEI	BOVESPA	EU	EM
2009-01-05	0.0357537	0.0383762	- 0.0046793	0.0021934	0.0038944	0.0000000	0.0311902	0.0126980	0.0285245
2009-01-06	0.0254259	0.0318127	0.0077867	0.0084553	0.0128656	0.0041625	0.0189196	0.0113407	0.0087726
2009-01-07	-	-	-	-	-	0.0172929	-	-	-
2009-01-08	0.0288617	0.0263530	0.0304691	0.0178331	0.0287346	-	0.0358986	0.0170728	0.0200154
2009-01-09	-	-	0.0033914	-	-	-	0.0282832	-	-
2009-01-12	0.0622081	0.0847159	-	0.0117263	0.0004660	0.0400613	-	0.0055610	0.0194238
2009-01-09	0.0098599	0.0096581	-	-	-	-	-	-	-
2009-01-12	-	-	0.0215332	0.0198728	0.0127097	0.0044735	0.0097639	0.0109886	0.0078022
2009-01-12	0.0291910	0.0423612	0.0228226	0.0135257	0.0050255	0.0490385	0.0538495	0.0124513	0.0226297

Now, let's convert the date column in our stock prices dataset into datetime:

```
stock_exchange$date <- as.Date(stock_exchange$date)
```

Now, as our `date` column already follows the **ISO 8601** format, we are quite fortunate. But, most of the times it will be difficult to acquire such well formatted data and we may end up getting a more complex time objects like **Monday, December 31, 2018 11:59:59 PM**. In this regard, the `lubridate` package allows much simpler conversion to a `POSIXct` object.

```
Date_lubri <- ymd(stock_exchange$date)

class(stock_exchange$date)
```

```
## [1] "Date"
```

```
class(Date_lubri)
```

```
## [1] "Date"
```

With this, you can now access a wide range of functionalities. The traditional `min` and `max` functions can also be used on the date columns.

```
min(stock_exchange$date)
```

```
## [1] "2009-01-05"
```

```
max(stock_exchange$date)
```

```
## [1] "2011-02-22"
```

```
cat(sprintf("
Date: %s,
Day of year: %s,
Day of month: %s,
Day of week: %s,
Name of Day: %s,
Month: %s,
Month Name: %s,
Quarter: %s,
Year: %s,
ISO Week: %s,
",
  stock_exchange$date[1], yday(stock_exchange$date[1]), mday(stock_exchange$date[1]),
  wday(stock_exchange$date[1], week_start = 1), weekdays(stock_exchange$date[1]),
  month(stock_exchange$date[1]), month(stock_exchange$date[1],
    label = TRUE, abbr = FALSE), quarter(stock_exchange$date[1]),
  year(stock_exchange$date[1]), isoweek(stock_exchange$date[1])))
```

```
##
## Date: 2009-01-05,
## Day of year: 5,
## Day of month: 5,
## Day of week: 1,
## Name of Day: Monday,
## Month: 1,
## Month Name: January,
## Quarter: 1,
## Year: 2009,
## ISO Week: 2,
```

Unlike `python`, we don't usually convert the date column of the dataframe to index. If we would want to conduct some time-series operations, we usually convert the dataframe to a time-series object.

```
require(xts)
```

```
stockexchange_xts <- xts(stock_exchange[, -which(names(stock_exchange) ==
  "date")], order.by = stock_exchange$date)
```

Now, using the date you can pull up lot of information you need. Let's try to get the values recorded during the month of January, 2010.

```
head(stockexchange_xts["2010-01"], 5)
```

```
##           ISE...2      ISE...3      SP      DAX      FTSE
## 2010-01-04 0.010229371 0.014477562 0.015916069 0.000000000 0.016017643
## 2010-01-05 0.013898022 0.024018915 0.003110806 -0.002721820 0.004028020
## 2010-01-06 0.007956979 0.005706348 0.000545376 0.000409408 0.001357159
## 2010-01-07 0.007771749 0.007498474 0.003993283 -0.002483888 -0.000596923
## 2010-01-08 -0.003189192 0.000835311 0.002877549 0.003027297 0.001356129
##           NIKKEI      BOVESPA      EU      EM
## 2010-01-04 0.000000000 0.000000000 0.016777763 0.008399010
## 2010-01-05 0.002534611 0.002780057 0.001315820 0.008066740
## 2010-01-06 0.004634515 0.006937723 0.000585708 0.005329620
## 2010-01-07 -0.004650430 -0.003938240 -0.000359589 -0.006374655
## 2010-01-08 0.010862314 -0.002672088 0.003554423 0.001229253
```

You can also explore for date range, month range and specify date after which you would like to select all the data. Basically with / operator you can filter your desired dataset.

```
head(stockexchange_xts["2009-01/05"], 5)
```

```
##           ISE...2      ISE...3      SP      DAX      FTSE
## 2009-01-05 0.035753708 0.038376187 -0.004679315 0.002193419 0.003894376
## 2009-01-06 0.025425873 0.031812743 0.007786738 0.008455341 0.012865611
## 2009-01-07 -0.028861730 -0.026352966 -0.030469134 -0.017833062 -0.028734593
## 2009-01-08 -0.062208079 -0.084715902 0.003391364 -0.011726277 -0.000465999
## 2009-01-09 0.009859905 0.009658112 -0.021533208 -0.019872754 -0.012709717
##           NIKKEI      BOVESPA      EU      EM
## 2009-01-05 0.000000000 0.03119023 0.012698039 0.028524462
## 2009-01-06 0.004162452 0.01891958 0.011340652 0.008772644
## 2009-01-07 0.017292932 -0.03589858 -0.017072795 -0.020015412
## 2009-01-08 -0.040061309 0.02828315 -0.005560959 -0.019423778
## 2009-01-09 -0.004473502 -0.00976388 -0.010988634 -0.007802212
```

The `ts_info` function from the `TSstudio` package provides a concise summary of most of the preceding functions. This includes the object class, the number of observations, the frequency, and the starting and ending of the series.

```
require(TSstudio)
```

```
ts_info(stockexchange_xts)
```

```
## The stockexchange_xts series is a xts object with 9 variables and 536 observations
## Frequency: daily
## Start time: 2009-01-05
## End time: 2011-02-22
```

You can also obtain a similar combination of the frequency, start, and end information by using the `periodicity` function

```
periodicity(stockexchange_xts)
```

```
## Daily periodicity from 2009-01-05 to 2011-02-22
```

The `is.regular` function can be used for identifying whether a `xts/zoo` object is regular or not. If the `strict` argument of the function is set to **TRUE**, the function will test whether the object is strictly regular.

```
is.regular(stockexchange_xts)
```

```
## [1] TRUE
```

Let us now look at the data type of the index of our `xts` object using the `class` function.

```
class(index(stockexchange_xts))
```

```
## [1] "Date"
```

`as.yearmon` and `as.yearqtr` are two conversion functions that allow us to convert `Date` objects to `yearmon` and `yearqtr` objects, respectively. This can help us in indexing a monthly and quarterly series. These functions are a part of the `zoo` package. Let's see an example:

```
monthly_dates <- seq.Date(from = as.Date("2019-01-01"), length.out = 12,  
  by = "month")  
head(monthly_dates)
```

```
## [1] "2019-01-01" "2019-02-01" "2019-03-01" "2019-04-01" "2019-05-01"  
## [6] "2019-06-01"
```

```
monthly_yearmon <- as.yearmon(monthly_dates)  
head(monthly_yearmon)
```

```
## [1] "Jan 2019" "Feb 2019" "Mar 2019" "Apr 2019" "May 2019" "Jun 2019"
```

Let's now apply it on our data:

```
head(as.yearmon(index(stockexchange_xts)), 5)
```

```
## [1] "Jan 2009" "Jan 2009" "Jan 2009" "Jan 2009" "Jan 2009"
```

```
head(as.yearqtr(index(stockexchange_xts)), 5)
```

```
## [1] "2009 Q1" "2009 Q1" "2009 Q1" "2009 Q1" "2009 Q1"
```

Apart from the above functions, The `xts` package provides us with a set of functions for handling the index of the `xts` object. The `indexClass` family of functions provides us with a set of tools for extracting or replacing the `xts` class index. The `convertIndex` function converts the class of the object index.

```
head(convertIndex(stockexchange_xts, "yearmon"), 5)
```

```
##           ISE...2      ISE...3      SP      DAX      FTSE  
## Jan 2009  0.035753708  0.038376187 -0.004679315  0.002193419  0.003894376  
## Jan 2009  0.025425873  0.031812743  0.007786738  0.008455341  0.012865611  
## Jan 2009 -0.028861730 -0.026352966 -0.030469134 -0.017833062 -0.028734593  
## Jan 2009 -0.062208079 -0.084715902  0.003391364 -0.011726277 -0.000465999  
## Jan 2009  0.009859905  0.009658112 -0.021533208 -0.019872754 -0.012709717
```



```
##           NIKKEI      BOVESPA          EU          EM
## Jan 2009  0.000000000  0.03119023  0.012698039  0.028524462
## Jan 2009  0.004162452  0.01891958  0.011340652  0.008772644
## Jan 2009  0.017292932 -0.03589858 -0.017072795 -0.020015412
## Jan 2009 -0.040061309  0.02828315 -0.005560959 -0.019423778
## Jan 2009 -0.004473502 -0.00976388 -0.010988634 -0.007802212
```

```
head(convertIndex(stockexchange_xts, "yearqtr"), 5)
```

```
##           ISE...2      ISE...3          SP          DAX          FTSE
## 2009 Q1  0.035753708  0.038376187 -0.004679315  0.002193419  0.003894376
## 2009 Q1  0.025425873  0.031812743  0.007786738  0.008455341  0.012865611
## 2009 Q1 -0.028861730 -0.026352966 -0.030469134 -0.017833062 -0.028734593
## 2009 Q1 -0.062208079 -0.084715902  0.003391364 -0.011726277 -0.000465999
## 2009 Q1  0.009859905  0.009658112 -0.021533208 -0.019872754 -0.012709717
##           NIKKEI      BOVESPA          EU          EM
## 2009 Q1  0.000000000  0.03119023  0.012698039  0.028524462
## 2009 Q1  0.004162452  0.01891958  0.011340652  0.008772644
## 2009 Q1  0.017292932 -0.03589858 -0.017072795 -0.020015412
## 2009 Q1 -0.040061309  0.02828315 -0.005560959 -0.019423778
## 2009 Q1 -0.004473502 -0.00976388 -0.010988634 -0.007802212
```

In addition, the `indexClass` function provides a set of functions for extracting the date or time component of the object index. For example, the `.indexwday` function returns the corresponding days of the week:

```
head(.indexwday(stockexchange_xts))
```

```
## [1] 1 2 3 4 5 1
```

Another useful function is `indexFormat`, which allows us to modify the index format. We will utilize the function to modify the index format from **year, month, day (y-m-d)** to **day, month, year (d-m-y)**:

```
indexFormat(stockexchange_xts) <- "%d-%m-%Y"
```

```
head(stockexchange_xts)
```

```
##           ISE...2      ISE...3          SP          DAX          FTSE
## 05-01-2009  0.035753708  0.038376187 -0.004679315  0.002193419  0.003894376
## 06-01-2009  0.025425873  0.031812743  0.007786738  0.008455341  0.012865611
## 07-01-2009 -0.028861730 -0.026352966 -0.030469134 -0.017833062 -0.028734593
## 08-01-2009 -0.062208079 -0.084715902  0.003391364 -0.011726277 -0.000465999
## 09-01-2009  0.009859905  0.009658112 -0.021533208 -0.019872754 -0.012709717
## 12-01-2009 -0.029191028 -0.042361155 -0.022822626 -0.013525735 -0.005025533
##           NIKKEI      BOVESPA          EU          EM
## 05-01-2009  0.000000000  0.03119023  0.012698039  0.028524462
## 06-01-2009  0.004162452  0.01891958  0.011340652  0.008772644
## 07-01-2009  0.017292932 -0.03589858 -0.017072795 -0.020015412
## 08-01-2009 -0.040061309  0.02828315 -0.005560959 -0.019423778
## 09-01-2009 -0.004473502 -0.00976388 -0.010988634 -0.007802212
## 12-01-2009 -0.049038532 -0.05384947 -0.012451259 -0.022629745
```

As the `xts` class is a **semi-data frame**, it is relatively simple to manipulate or extract data by using a `data.frame` style. Moreover, it is possible to retrieve data based on a specific time range using the series index. For example, let's say that you wish to extract the first 12 observations of the FTSE data from the stock price dataset that we created.

```
FTSE_xts1 <- stockexchange_xts$FTSE[1:12]
ts_info(FTSE_xts1)
```

```
## The FTSE_xts1 series is a xts object with 1 variable and 12 observations
## Frequency: daily
## Start time: 2009-01-05
## End time: 2009-01-20
```

The xts object supports text-based queries for subsetting the series. For instance, we can extract the first year of the series by specifying the year value (that is, 2009):

```
FTSE_xts2 <- stockexchange_xts$FTSE["2009"]
ts_info(FTSE_xts2)
```

```
## The FTSE_xts2 series is a xts object with 1 variable and 250 observations
## Frequency: daily
## Start time: 2009-01-05
## End time: 2009-12-31
```

Furthermore, it is possible to have a more granular subset by using other elements of the index object. For instance, if you wish to extract the first 6 months of the year 2009, you can use the following structure:

```
FTSE_xts3 <- stockexchange_xts$FTSE["2010-01/"]
ts_info(FTSE_xts3)
```

```
## The FTSE_xts3 series is a xts object with 1 variable and 286 observations
## Frequency: daily
## Start time: 2010-01-04
## End time: 2011-02-22
```